

# Emulador de File System Virtual con Asignación Enlazada

**Curso:** Sistemas Operativos (Escuela de Computación)  
**Institución:** Instituto Tecnológico de Costa Rica (ITCR)  
**Profesora:** Ing. Erika Marín Schumann  
**Semestre:** I Semestre 2026

## Integrantes

| Nombre                      | Carnet     |
|-----------------------------|------------|
| Daniel Alemán Ruiz          | 2023051957 |
| Joseph Arrieta Mora         | 2023020875 |
| Sebastián Rodríguez Sánchez | 2023074446 |

## Descripción General

Este proyecto implementa un **emulador de sistema de archivos (File System)** virtual con estructura jerárquica de directorios y almacenamiento respaldado en un archivo binario (**disk.bin**) que actúa como el disco físico simulado.

El sistema emplea una **Tabla de Asignación de Archivos (FAT)** gestionada en memoria, con asignación de sectores mediante el algoritmo **First Fit** y encadenamiento de bloques de forma **enlazada**, lo que permite fragmentación no contigua. La interfaz gráfica, construida con **Flet**, ofrece una terminal interactiva con árbol de directorios dinámico y visualización de la ruta actual en todo momento.

## Requisitos de Diseño Implementados

| Requisito                  | Estado          | Mecanismo                                                                                                                             |
|----------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Asignación Enlazada        | ✓<br>Completado | La FAT en memoria encadena sectores: cada sector apunta al siguiente o a EOF ( <b>-2</b> ).                                           |
| Algoritmo First Fit        | ✓<br>Completado | <b>find_free_sectors</b> escanea linealmente la FAT desde el sector <b>0</b> hasta encontrar la cantidad requerida de bloques libres. |
| Control de Sectores Vacíos | ✓<br>Completado | La FAT se inicializa con <b>-1</b> (libre). Al liberar un archivo, se restauran los sectores a <b>-1</b> .                            |
| Almacenamiento No Contiguo | ✓<br>Completado | Los sectores asignados no necesitan ser contiguos; los punteros de la FAT los encadenan.                                              |

| Requisito                       | Estado                           | Mecanismo                                                                                                                                                          |
|---------------------------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Persistencia Física             | <div><div></div>Completado</div> | Los datos se escriben en <code>file-system/disk.bin</code> . Al cerrar la aplicación, el archivo binario persiste pero la jerarquía lógica en memoria se reinicia. |
| Seguridad de Nombres Duplicados | <div><div></div>Completado</div> | Si el nombre ya existe, se lanza <code>FileExistsConflictException</code> y se solicita confirmación de sobrescritura.                                             |
| Aviso de Disco Lleno            | <div><div></div>Completado</div> | Se lanza <code>DiskFullException</code> si no hay sectores libres suficientes para el archivo nuevo.                                                               |

## Estructura del Proyecto

```
/flet-file-system
├── main.py           # Interfaz gráfica (Flet) y dispatcher de comandos
├── README.md         # Este archivo
├── /core
│   ├── file_system.py # Lógica del FS: FSNode, FileSystem, comandos
│   └── virtual_disk.py # Disco virtual: FAT, First Fit, lectura/escritura
├── /file-system
│   └── disk.bin       # Archivo binario del disco simulado (generado en
runtime)
└── /assets
    └── fs_icon.png    # Ícono de la aplicación
```

## Arquitectura y Componentes

### 1. Interfaz Gráfica (`main.py`)

Desarrollada con **Flet** bajo un diseño moderno de alta fidelidad:

- **Árbol de Directorios (TREE):** Panel lateral izquierdo con visualización en tiempo real de la jerarquía completa. Resalta visualmente el directorio actual (➔), diferencia carpetas (📁) de archivos (📄) y usa colores distintos por tipo.
- **Terminal Integrada:** Soporta comandos interactivos con atajos de teclado estilo Unix:
  - Historial de comandos con ⬆ / ⬇.
  - `Ctrl+U` para limpiar la línea, `Ctrl+W` para borrar la última palabra, `Ctrl+K` para truncar.
  - Autocompletado de rutas virtuales con `Tab`.
- **Ruta Actual Dinámica:** Visible en el encabezado en todo momento.
- **Diálogos Gráficos:** Confirmación de sobrescritura, editor de archivos (FILE/MODFILE) con validación inline.

### 2. Disco Virtual (`core/virtual_disk.py`)

Módulo de abstracción a bajo nivel:

- **create\_disk(num\_sectors, sector\_size)**: Crea físicamente el archivo `disk.bin` con `num_sectors` × `sector_size` bytes en ceros e inicializa la FAT.
- **read\_sector(i) / write\_sector(i, data)**: Operaciones atómicas de I/O por bloque.
- **find\_free\_sectors(n)**: Implementación de **First Fit** — escanea secuencialmente y retorna los primeros `n` sectores libres (`fat[i] == -1`).
- **allocate\_file\_sectors(size\_bytes)**: Calcula sectores necesarios, los busca con First Fit y los enlaza en la FAT.
- **free\_file\_sectors(start)**: Libera la cadena entera de sectores de un archivo.
- **get\_free\_sectors\_count()**: Retorna el total de sectores disponibles.

### 3. File System Lógico (`core/file_system.py`)

Capa de alto nivel:

- **FSNode**: Nodo del árbol (archivo o directorio). Almacena nombre, padre, hijos, `first_sector`, `size`, fechas de creación y modificación.
- **FileSystem**: Raíz del árbol y directorio actual. Implementa todos los comandos del sistema.
- **Resolución de rutas**: Soporte para rutas absolutas (`/dir/subdir`), relativas (`., ..`) y el alias `/root/....`
- **Excepciones**: `FileSystemException`, `DiskFullException`, `FileExistsConflictException`.

---

## Comandos Disponibles

| Comando               | Sintaxis                                                | Descripción                                                                 |
|-----------------------|---------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>CREATE</b>         | <code>CREATE &lt;sectores&gt;<br/>&lt;tamaño&gt;</code> | Inicializa el disco virtual.                                                |
| <b>FILE</b>           | <code>FILE [nombre.ext]</code>                          | Crea un archivo (abre el editor gráfico).                                   |
| <b>MKDIR</b>          | <code>MKDIR &lt;ruta&gt;</code>                         | Crea un directorio (soporta rutas anidadas).                                |
| <b>CAMBIARDIR</b>     | <code>CAMBIARDIR &lt;ruta&gt;</code>                    | Cambia el directorio actual ( <code>., ..</code> , absoluto, relativo).     |
| <b>LISTARDIR</b>      | <code>LISTARDIR [ruta]</code>                           | Lista el contenido del directorio actual o de la ruta indicada.             |
| <b>MODFILE</b>        | <code>MODFILE &lt;ruta&gt;</code>                       | Modifica el contenido de un archivo (abre el editor gráfico).               |
| <b>VERFILE</b>        | <code>VERFILE &lt;ruta&gt;</code>                       | Muestra el contenido de un archivo en la terminal.                          |
| <b>VERPROPIEDADES</b> | <code>VERPROPIEDADES &lt;ruta&gt;</code>                | Muestra nombre, extensión, tamaño, fecha de creación y modificación.        |
| <b>COPY</b>           | <code>COPY &lt;origen&gt;<br/>&lt;destino&gt;</code>    | Copia archivos/directorios: Real→Virtual, Virtual→Real, Virtual→Virtual.    |
| <b>MOVER</b>          | <code>MOVER &lt;origen&gt;<br/>&lt;destino&gt;</code>   | Mueve o renombra archivos/directorios (funciona como <code>rename</code> ). |

| Comando       | Sintaxis                                       | Descripción                                                  |
|---------------|------------------------------------------------|--------------------------------------------------------------|
| <b>REMOVE</b> | <code>REMOVE &lt;ruta&gt; [ruta2 ...]</code>   | Elimina archivos o directorios (recursivo para directorios). |
| <b>FIND</b>   | <code>FIND &lt;patrón&gt; [ruta_inicio]</code> | Busca por nombre (soporta comodines * y ?).                  |
| <b>TREE</b>   | <code>TREE</code>                              | Imprime el árbol completo del File System en la terminal.    |
| <b>MAPA</b>   | <code>MAPA</code>                              | Muestra el estado actual de la FAT (sectores libres/usados). |
| <b>HELP</b>   | <code>HELP</code>                              | Lista todos los comandos con su descripción.                 |

## Instrucciones de Ejecución

### Prerrequisitos

Python 3 y la biblioteca `flet`:

```
pip install flet
```

### Iniciar la Aplicación

Desde la raíz del proyecto:

```
python main.py
```

### Flujo Básico

```
# 1. Inicializar el disco virtual (5000 sectores de 512 bytes = ~2.4 MB)
CREATE 5000 512

# 2. Crear directorios
MKDIR documentos
MKDIR documentos/fotos

# 3. Crear archivos (abre el editor gráfico)
FILE documentos/notas.txt

# 4. Navegar
CAMBIARDIR documentos
LISTARDIR

# 5. Ver propiedades y contenido
```

```
VERPROPIEDADES notas.txt
VERFILE notas.txt

# 6. Copiar un archivo real al FS virtual
COPY C:\Users\usuario\archivo.txt /documentos/archivo.txt

# 7. Buscar archivos
FIND *.txt

# 8. Ver árbol completo
TREE
```

---

## Notas de Diseño

- **Fragmentación interna:** Al ser asignación enlazada, el último sector de un archivo puede estar parcialmente vacío (relleno con `\x00`).
- **Persistencia:** Al cerrar la app, el árbol lógico en memoria se pierde, pero `disk.bin` permanece en disco.
- **Ruta virtual siempre visible:** El encabezado de la interfaz muestra la ruta actual en todo momento.
- **Comodines en FIND:** Se soportan patrones como `*.txt`, `nota?`, `doc*`.
- **MOVER como rename:** `MOVER archivo.txt nuevo_nombre.txt` renombra el archivo en el mismo directorio.